
Team Coordination, Project & Issue Management

Oscar Chaparro

Agenda

1 Team Coordination — Roles, responsibilities & communication

2 Project Management — Decomposition, estimation & tracking

3 Issue Management — Organizing, triaging & tracking

How work moves



Coordination and constraints

INDUSTRY

**Set for
teams**

Contract signed

Regulatory deadline

Delivery date negotiated
with the customer

Team composition

**Managed by
teams**

Sprint 1

Sprint 2

Sprint 3

Sprint 4

Sprint 5

...

TYPICALLY DICTATED

- Delivery dates — contracts, regulation, customer negotiation
- Who is on the team, and for how long
- Process and tooling, in many organizations

ACTUALLY YOURS

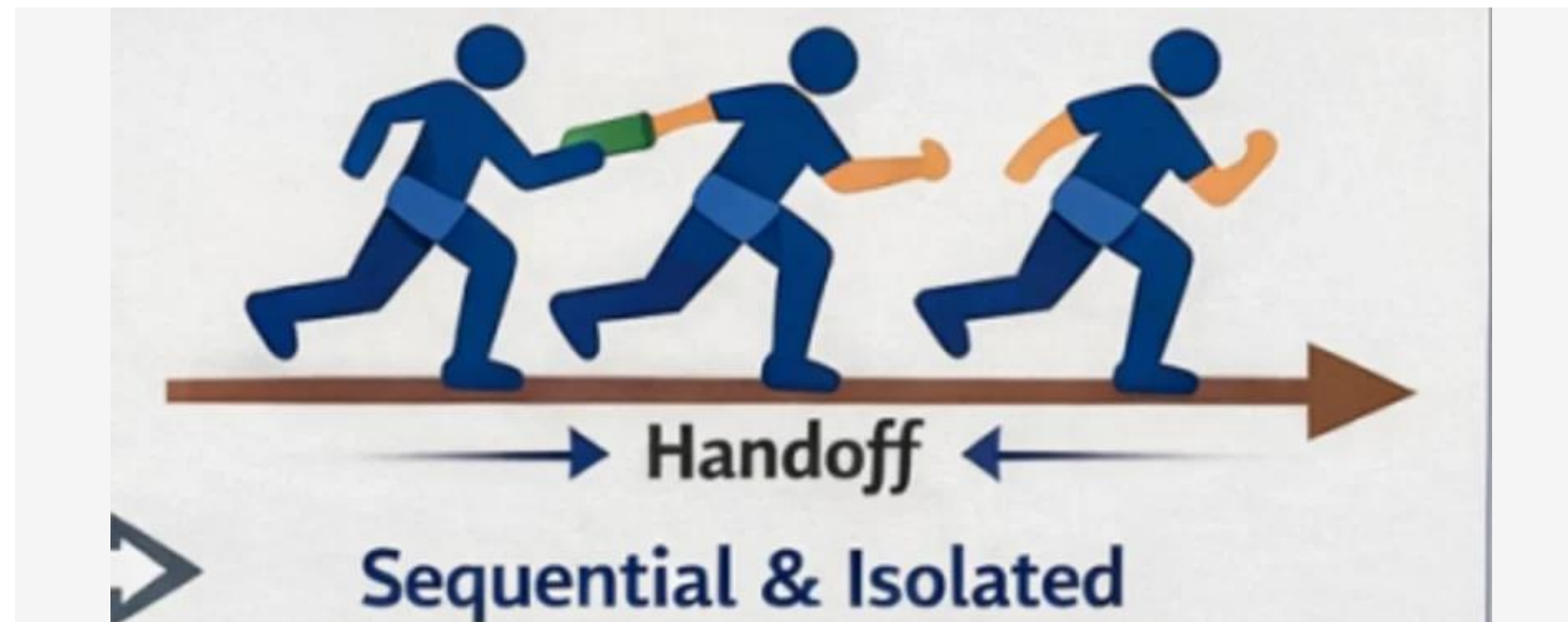
- How work is decomposed and sequenced
- How it is tracked and made visible
- How it is reviewed and handed off

Team Coordination

Roles, Responsibilities & Communication

Relay race or rugby?

INDUSTRY



Waterfall — a relay race

The baton is passed sequentially. Each runner works alone.



Modern software — rugby

Cross-functional · self-organizing · overlapping phases

Takeuchi & Nonaka, *The New New Product Development Game*, HBR 1986

Scrum roles at a glance

YOUR PROJECT

TODAY

Roles

Product Owner ScrumMaster
Development Team

Each role is a distinct accountability
— not a job title.

SEE LAST LECTURE

Ceremonies

Sprint planning Daily
stand-up Review &
retrospective

SEE LAST LECTURE

Artifacts

Product backlog Sprint
backlog Increment

The development team is self-organizing — nobody in this picture assigns work.

Product Owner

- Maximizes **return on investment (ROI)** — value delivered against what the work cost
- Owns and orders the product backlog
- Ensures the team understands backlog items
- **One person — not a committee**

ScrumMaster

- **Servant-leader** — not a manager or task assigner
- Ensures Scrum is understood and enacted
- Removes impediments blocking the team
- Coaches the team and the organization

Does not assign tasks and does not decide what gets built.

The Development Team

YOUR PROJECT

- **Cross-functional:** all skills to deliver a “Done” increment
- **Self-organizing:** no titles, no sub-teams, no hierarchy
- **Size:** typically ten or fewer · full-time commitment
- **Collectively accountable** for the sprint goal

Design

Code

Test

Deploy

Brooks's law

INDUSTRY

Adding people to a late project makes it later.

3 PEOPLE

3

communication paths

10 PEOPLE

45

paths — worst case, everyone to everyone

$$n(n - 1) / 2$$

Paths grow quadratically. Onboarding cost is paid by the people already working.

- **Team size:** small teams answer superlinear coordination cost
- **Scaling:** ownership and interfaces remove dependencies; frameworks only manage them
- **Planning:** with a fixed date, adding people is not a real lever. Scope is.

Roles / stakeholders

YOUR PROJECT

Inside the team

Developers* Technical lead* QA /
test engineers* Product owner
Business/reqs analysts Architects
UI/UX designers DevOps · Site
Rel. Eng. / on-call Data engineers ·
DB Admins

Inside the company, outside the team

Project / delivery manager Security
specialists Release / program
managers Support & customer
success Legal, compliance,
procurement

Outside the company

The customer who pays End users
(often not the customer) Regulatory
bodies Auditors & certification
authorities Vendors and integration
partners

Work and roles beyond Scrum

INDUSTRY

METHODOLOGY	HOW WORK IS SCHEDULED	KEY ROLES	WHERE IT FITS
Scrum	Fixed-length sprints	Prod Owner (sets priority) · Scrum Master · Devs	Stable teams, negotiable scope
Kanban	Continuous flow, WIP limits, pull	No prescribed roles · the queue owner sets priority	Support and ops; work arrives unpredictably
XP (Extreme Prog)	Short iterations plus engineering practices	On-site Customer (sets priority) · Devs in pairs · Coach	Teams that need pairing, TDD, CI, collective ownership
Waterfall / stage-gate	Phases with sign-off gates	Proj Mgr · Analyst · Architect · Devs · QA	Regulated, safety-certified or fixed-price work
SAFe (Scaled Agile Framework)	Quarterly planning across a release train	Release Train Eng · Prod Mgmt · team Prod Owner + Scrum Master	Many teams delivering one product

The method is usually chosen by constraints, not by preference.

Team working agreement

YOUR PROJECT

Decide how the team operates on day one. Keep it in the repo. Revisit it at the retro.

Availability

When each person works, expected response time, and what counts as urgent

Where decisions live

If it is not in the issue or the PR, it did not happen

Definition of done

Tests pass, reviewed, merged, issue closed

Review turnaround

A committed maximum — one working day

Meeting cadence

What, when, how long, and what happens if you miss one

Escalation

What the team does when someone goes quiet

Conflict is expected. Set the rules while it is cheap.

Sprint 0 covered some of this. Real development starts now: agree to it explicitly.

Leader vs. Manager

YOUR PROJECT

Leader

Sets vision and direction
Shapes team culture
Leads by example, earns trust
Inspires rather than controls

Manager

Makes work assignments and schedules
Finds and allocates resources
Gives team members autonomy
Facilitates communication

Technical lead

Architecture and technical decisions. Leads through credibility and judgement.

Project / delivery manager

Schedule, scope, reporting, external commitments. Coordinates through process.

Product owner

What gets built and in what order.

Leadership without a title

YOUR PROJECT

In your team, nobody has authority. Leadership is behavior.

- Who calls the meeting when the team has drifted
- Who unblocks the teammate who has been stuck for three days
- Who notices the integration risk before the last week
- Who writes down the decision everyone thinks they remember
- Who says the uncomfortable thing in the retro

On your team: rotate the role weekly. The person on duty runs stand-up, grooms the board, and runs the retro.

Resolving team conflicts

YOUR PROJECT

Conflict is normal — it usually peaks early, before the team has agreed how it works.

- Goal: find a solution and move forward
- Ensure everyone works from the same set of facts
- Establish discussion ground rules before starting
- Express how you feel — never presume others' intent
- Stay calm; if triggered, step away briefly and return
- If impasse: escalate to a team leader or facilitator

“Fight forward, not back.”

Unresolved conflict
stalls the team.

The daily stand-up

Time-boxed: **15 minutes** · same time and place every day · the team runs it, not the ScrumMaster

1

What did I complete since yesterday?

2

What will I do today?

3

What obstacles are blocking me?

 Status report for a manager

 **Peer commitment to the team**

Sprint review & retrospective

YOUR PROJECT

Sprint Review

EXTERNAL FOCUS

Team demos completed work to stakeholders
Stakeholders provide feedback PO updates the backlog from that feedback

Time-boxed: 1–2 hours

Sprint Retrospective

INTERNAL FOCUS

START — what should we start doing?

STOP — what should we stop doing?

CONTINUE — what is working?

Team-owned; ScrumMaster facilitates. Output: concrete improvement actions.

How to run a meeting

YOUR PROJECT

1

Set the agenda

Share it before the meeting — topics, owners, time slots

2

Start on time, end on time

Respect everyone's schedule

3

End with action items

What, who, by when — captured in GitHub Issues or notes

Coordinator

Runs the meeting, keeps it on track

Scribe

Captures notes and action items

Checker

Signals when discussion drifts off-topic

Do you need a meeting?

YOUR PROJECT

Why do you want to call a meeting?

Complex discussion · decision
needs real-time alignment



Schedule a meeting

Information sharing · simple status
update



**Send an async message or
email**

Decision or feedback that must be
tracked

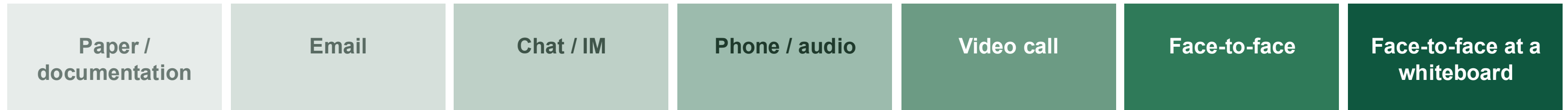


**File a GitHub issue or
comment**

Meetings are expensive — default to async when you can.

Communication channels

YOUR PROJECT



Least effective

Most effective →

Rule: match channel richness to task complexity.

- Casual status update → chat is fine
- Architectural decision → face-to-face or video

An argued heuristic, not a measurement. Cockburn and Ambler drew this from practitioner observation. Ambler adds a second axis: the richest channels leave no record.

It predates distributed work. Written async communication is durable, searchable, timezone-independent and forces clarity — the model scores all of that as loss.

Pick the channel(s) for each.

Docs

Email

Chat

Phone

Video

Face-to-face

Whiteboard

- A design decision the team will need to look up in three weeks
- Telling a teammate their share of the work isn't landing
- A bug you can't reproduce and need a second pair of eyes on
- Deciding what to cut two days before the demo
- Onboarding a teammate to code you wrote

Scaling: Scrum of Scrums

INDUSTRY

Scrum of Scrums

one ambassador per team · daily



Team A

full Scrum

Team B

full Scrum

Team C

full Scrum

Three questions at the sync

- What did my team do that affects yours?
- What will we do next?
- What cross-team blockers exist?

The ambassador attends the sync; the rest of each team stays focused.

Conway's law

INDUSTRY

A system's structure mirrors the communication structure of the organization that built it.

Functional teams → one monolith

Frontend team

Backend team

Database team

One deployable, three owners. Every feature crosses all three teams.

Vertical teams → modules or services

Checkout UI · API ·
data

Search UI · API · data

Accounts UI · API ·
data

One owner each. Most features live inside one team; coordination only at the seams.

It has been measured. Microsoft studied Windows Vista: how many engineers touched a component, and how many orgs they spanned, predicted failure-prone code better than complexity or churn did.

On your team: try to give each person a module to own, not tasks from a shared pile. Agree the interface at every boundary before either side builds.

Amazon two-pizza teams

INDUSTRY

- **Small teams:** small enough to be fed with two pizzas
- **Service ownership:** one team owns one service end to end — build, deploy, operate
- **The 2002 API mandate:** all inter-team communication goes through service interfaces
- **“You build it, you run it”:** the team that ships the code carries the pager

Catalog

Cart

Checkout

Payments

Search

Recommendations

Reviews

Shipping

WHY IT WORKS

Teams agree on an interface once, then build without meeting together. Most communication paths stop existing instead of having to be managed.

WHAT IT COSTS

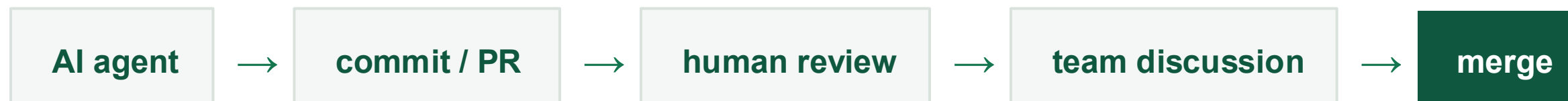
Every team runs its own service: deploys, monitoring, on-call. Duplication across teams (their own API, DB, etc.) is accepted as the price of independence.

AI in team coordination

INDUSTRY

Emerging reality: AI agents as active team participants.

- Agents can own backlog items, open PRs, and respond to issues autonomously
- **Accountability:** AI-generated work must still map to a human owner
- Humans must understand, review, and vouch for outputs before they merge
- **Risk:** AI produces work faster than the team can review it → invisible review bottlenecks



Treat AI like a capable junior team member — fast and useful, but requiring oversight and explicit handoffs.

Project Management

Decomposition, Estimation & Tracking

Iterative planning

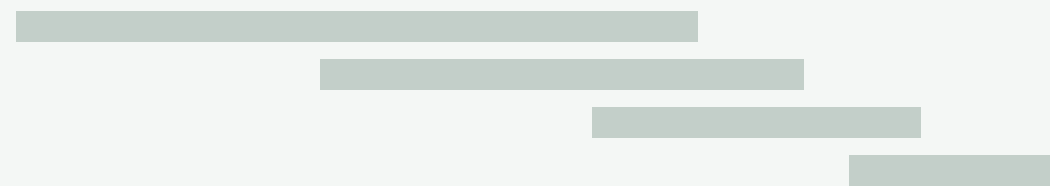
INDUSTRY

⚠ NOT OUR FOCUS TODAY

Plan-driven

Gantt · PERT · CPM

Full scope defined upfront Detailed schedule down to individual tasks Changes are expensive and require replanning Works when requirements are stable



WHAT WE USE

Iterative / Agile

Sprints · backlog · velocity

Scope emerges sprint by sprint Only the near horizon is planned in detail Changes are expected — reordering the backlog is cheap Works when requirements evolve

How planning actually works

YOUR PROJECT

Year

Budget, headcount, a few committed objectives

Decided by: Executives, finance

Quarter

Roadmap themes and epics, rough sizing, cross-team dependencies

Decided by: Project / program management with team input

Sprint

Detailed decomposition and estimation of two weeks of work

Decided by: The team

WHAT DRIVES THE PLAN

- Fixed external dates are inputs, not outputs
- Capacity is deliberately underplanned — unplanned work is guaranteed
- Cross-team dependencies — where most quarterly plans fail

Most projects overrun. A review of estimation surveys found schedule or budget overruns in 60–80% of projects. Re-planning each quarter is the mechanism, not a failure of the last plan.

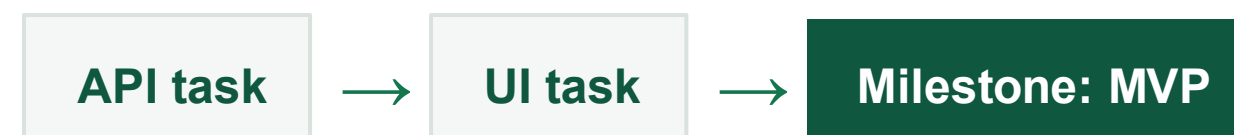
Milestones & task dependencies

YOUR PROJECT

Milestones mark significant deliverables or decision points
— “MVP complete,” “Beta release,” “Sprint 2 done.”

Task dependencies: some tasks cannot start until others finish — this shapes sprint planning.

In GitHub, milestones group issues by release or sprint;
progress shows as the percentage of closed issues.



What to read off a milestone: name · due date · open/closed issue count · progress bar

Work decomposition

YOUR PROJECT

Serving the product vision:

Epic — Account management

Feature — User Authentication

User Story — “Log in with email”

Task — login form UI

Task — auth API endpoint

Task — unit & integration tests

Why decompose?

- Makes work estimable and assignable
- Enables parallel work across the team
- Gives visibility into progress at multiple levels

In your project everything is a **GitHub issue** . Most issues are stories; split one into task issues when it is too big for one person.

User stories

YOUR PROJECT

A user story captures **what** a user wants and **why** — not how it is built.

GOOD — ACCOUNTING SOFTWARE

“As an **accountant**, I want to **see all outstanding invoices** so that **I know what payments are owed** .”

FORMAT

As a **[role]** , I want **[function]** , so that **[value]**.

X “As a developer, I want to refactor the database layer.” — **no user value expressed; this is a task, not a story.**

User story format & the 3 C's

YOUR PROJECT

As a [role]

Who benefits? The user — not the developer, unless the developer *is* the user

I want [function]

What capability? Behavior-level, not implementation

So that [value]

Why does this matter? It justifies building it

The 3 C's — Ron Jeffries

Card

The written story — a reminder to have a conversation

Conversation

The discussion that fleshes out the details

Confirmation

Acceptance criteria that define “done”

INVEST criteria

YOUR PROJECT

- I Independent** — developable without depending on other stories
- E Estimable** — the team can size the effort; enough detail exists
- N Negotiable** — not a contract; details stay open until the sprint starts
- S Small** — fits in a sprint; completable in a few days
- V Valuable** — delivers value to an end user or customer
- T Testable** — acceptance criteria exist; the team knows when it's done

If a story fails any criterion — refine it or split it.

Repair this story.

“As a developer, I want to refactor the database layer so that the code is cleaner.”

Independent

Negotiable

Valuable

Estimable

Small

Testable

Check it against all six. Which does it pass?

Story points

YOUR PROJECT

- Capture complexity + uncertainty + volume in one number
- Relative sizing: story A is twice as complex as story B
- Avoids the false precision of hour estimates
- A point is meaningful **within** a team — never comparable across teams

COMMON SCALES



Fibonacci, or T-shirt sizes (S / M / L / XL)

EXAMPLE — HOTEL RESERVATION BACKLOG

Story	Points
Search availability	3
Book a room	5
Cancel booking	3
Payment integration	8



What do you estimate in?

YOUR PROJECT

UNIT	WHERE IT IS USED
T-shirt sizes	S / M / L / XL — epics and roadmap items, where precision is pointless
Story points	1, 2, 3, 5, 8, 13 — non-linear, because uncertainty grows with size
Ideal days	“How long uninterrupted” — intuitive, but read as calendar days
Story counting	Count completed stories instead. Needs consistent sizing
Hours	Tasks only — 2h, 4h, 1d. Never stories

A story at 13+ is a signal to split, not an estimate. Velocity turns points back into a forecast — within one team only.



CARD · CONVERSATION · CONFIRMATION

Card	Issue title and a short body
Conversation	The issue thread — and the stand-up
Confirmation	Acceptance criteria as a checklist

An issue has unlimited space, so teams write specs and skip the conversation. The card was small on purpose.

- **Label the type:** `bug` · `feature-request` · `enhancement` · `tech-debt` · `task`
- **Link the hierarchy:** sub-issues, or a task list (`- [ ] #42`)
- **Estimate stories only** — add a Points field on the board
- **Milestones = sprints** — a progress bar for free
- **Close via PR:** `Closes #42` links the code to the issue

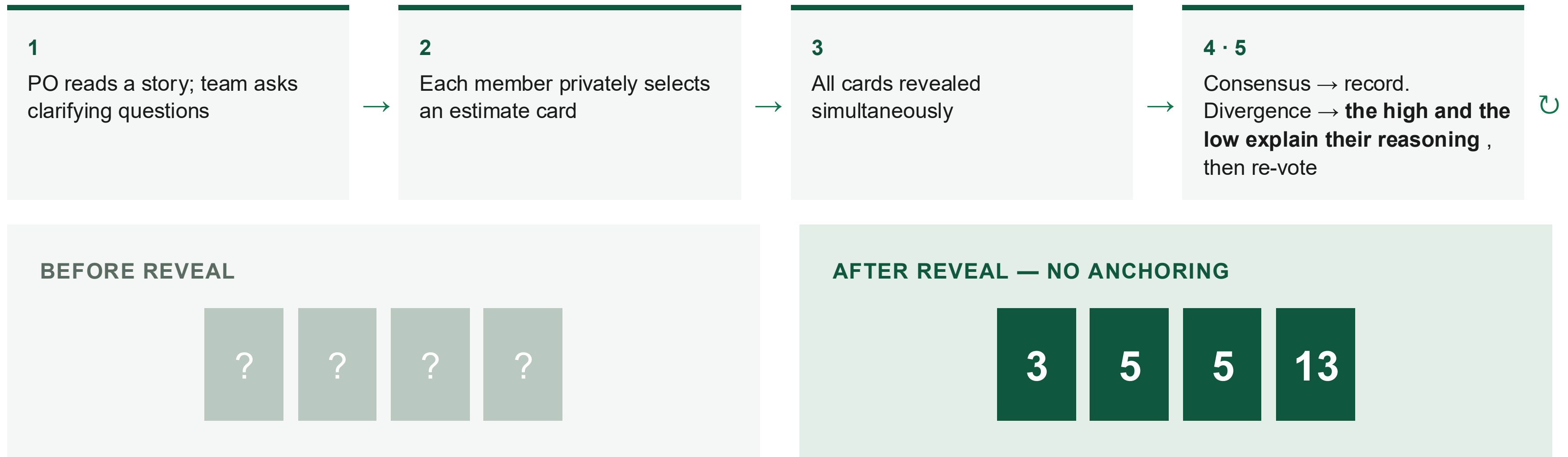
⚠ One issue per commit

That turns the tracker into a changelog. Split a story into tasks only when it needs more than one person, or more than a couple of days.



Planning Poker

YOUR PROJECT



The estimate is the byproduct. The product is shared understanding — if everyone agrees on the first reveal, the session is not earning its cost.

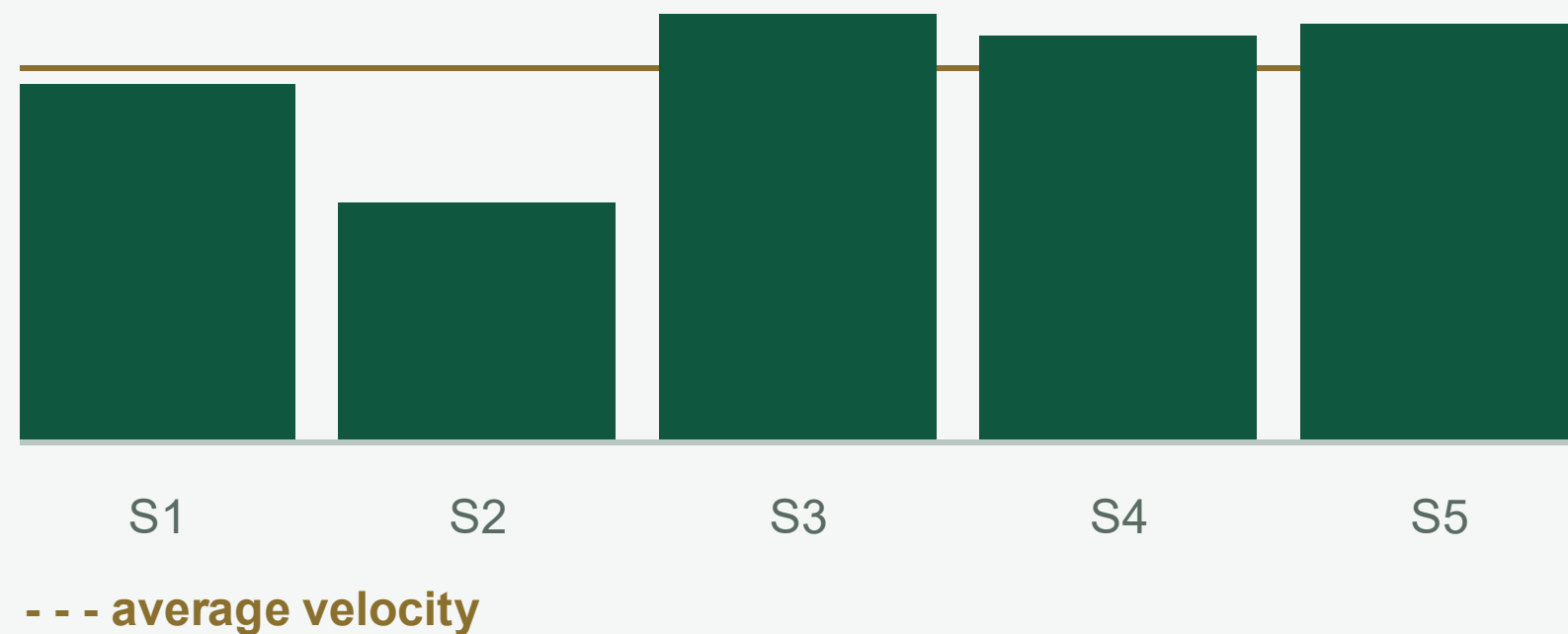
EXERCISE 3 MIN

One live round. Estimate “add password reset to the course project”. Reveal together, then the high and the low explain.

Velocity

INDUSTRY

POINTS COMPLETED PER SPRINT



Velocity = story points completed per sprint.

- Emerges after 2–3 sprints. You have none until sprint 1 closes — calibrate against a reference item until then
- Once stable, forecasts releases: $80 \div 20 = 4$ sprints — treat it as a range, not a date

⚠ Not a performance metric

Do not compare across teams or use it to pressure a team. Velocity is self-reported in a self-defined unit: the moment it is used to judge a team, the points inflate and the measure stops tracking anything. Rising velocity is recalibration, not improvement.

Measuring the system, not the people

INDUSTRY

“When a measure becomes a target, it ceases to be a good measure.”

Goodhart's law, as
phrased by
Strathern

Measure the system — quantitatively

- **Cycle time** — start to delivered. Report the 85th percentile, not the mean
- **Work in progress** — cycle time = WIP ÷ throughput. Start less to finish sooner
- **Escaped defects** — bugs found after release
- **DORA** — deploy frequency, lead time, change failure rate, time to restore

Assess people — qualitatively

- Impact on outcomes, not volume of output
- Peer feedback from the people who work with them
- Growth against explicit expectations for the level
- Judgement in reviews, design work and incidents

Lines of code, commits and tickets closed are trivially gamed, and they punish reviewing, mentoring and unblocking.

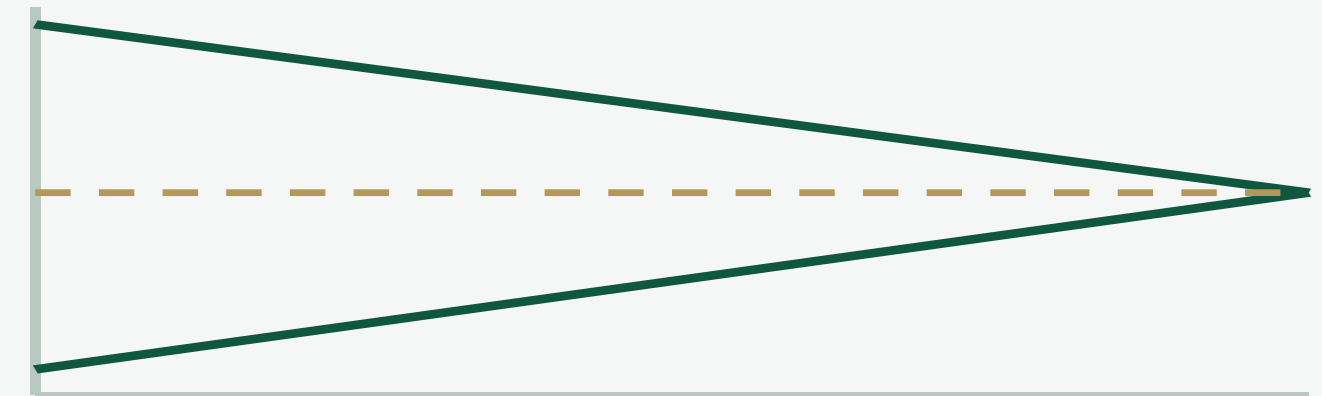
Rank individuals by system metrics and you break both.

Estimation is hard — honest caveats

INDUSTRY

- Early estimates are expected to be wrong — calibrate as you learn
- Estimates are forecasts, not commitments — give a range, not a date
- The goal of estimation is a rough forecast, not a commitment
- Accuracy improves as the team matures and requirements stabilize
- When in doubt: over-estimate rather than under-estimate — protect team capacity
- Use the retrospective to recalibrate systematically

CONE OF UNCERTAINTY



Estimate accuracy narrows as the project progresses and knowledge accumulates.

The product backlog

YOUR PROJECT

An **ordered** list of everything that might be built in the product.

- Owned and ordered by the Product Owner
- Never complete — continuously refined (“groomed”)
- Top items: detailed, estimated, ready to sprint
- Lower items: coarse, may lack estimates
- Backlog refinement: a regular session to break down and estimate upcoming items

HOTEL RESERVATION SYSTEM — PRODUCT BACKLOG

Story	Priority	Points	Sprint
✓ Search availability	P1	3	1
✓ Book a room	P1	5	1
✓ Cancel booking	P1	3	1
Payment integration	P2	8	2
Loyalty points	P3	—	—
Multi-language UI	P4	—	—

✓ = **sprint-ready: detailed, estimated, ordered at the top**

P1 now · P2 this sprint · P3 a coming sprint · P4 someday



Ordering the backlog

YOUR PROJECT

Set for you — assigned

From the instructor and TA. The must-do floor, roughly in the order given.

Managed by you — team

Tech debt, tests, tooling, refactors, bugs you found. You decide where they slot in.

WHAT DETERMINES THE ORDER

- **Dependencies first** — if B needs A, A is above B
- **Fixed dates** — anything tied to a course deadline
- **Value or risk** — highest uncertainty early, while you can react
- **Size** — a tiebreaker only

HOW IT RUNS

- One named person per sprint owns the order
- One list, ordered before planning ends
- **Ready before pullable** — description, acceptance criteria, no blocker
- Re-order at stand-up when reality changes

If pulling the next item requires a conversation, the backlog was not ordered.

Sprint planning

YOUR PROJECT

INPUTS

Product backlog (ordered)
Team capacity Previous
velocity



SPRINT PLANNING MEETING

PO presents top items Team estimates and
verifies Team selects items Tasks identified Hours
estimated per task

Time-boxed: 2 hours per sprint week (4 hours for
a 2-week sprint)



OUTPUTS

Sprint goal Sprint backlog

Running your sprint

YOUR PROJECT

The floor (announced on the assignment page)

A minimum per person, not the plan. Plan above it.

Calibrate “medium” first

Pick one change everyone understands, call it medium, size the rest against it. No points yet — you have no velocity.

- **Sequence, don’t just fill.** Check which changes touch the same files, and who merges first
- **One item in flight per person.** Finish before starting
- **Spare capacity pulls from the top,** never from what looks interesting
- **Everything is an issue,** setup included. Work off the board is invisible
- **Uneven contribution is fine** if everyone clears the floor and the board shows it
- **Integration cutoff 2–3 days before sprint end.** Nothing new starts; open work merges or drops

Count what actually completed. That is your first velocity — sprint 2 plans against it, not the floor.

Sprint planning: story → tasks with hours

YOUR PROJECT

USER STORY

Vacation planner

5 story points



Abstract points become trackable daily tasks with hour estimates.

1 · Design UI mockup

4 h

2 · Implement date picker component

6 h

3 · Connect to availability API

8 h

4 · Write unit tests

3 h

5 · User acceptance testing

2 h

Total

23 h



No changes during a sprint

YOUR PROJECT

 IN THIS SPRINT — FROZEN

Sprint backlog

Committed at planning. No additions, no swaps.

NEXT SPRINT — OPEN

Product backlog

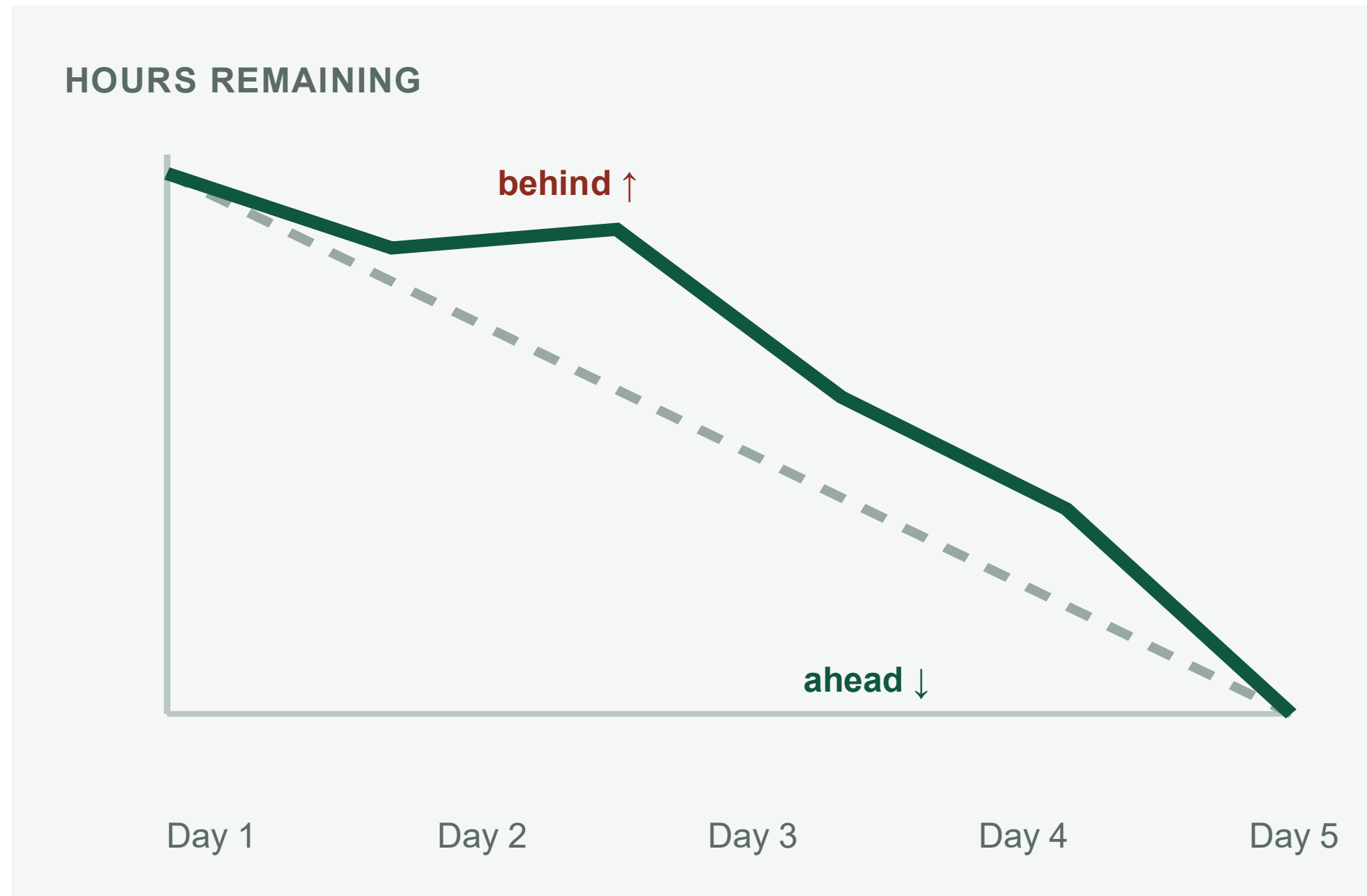
New requests land here and are considered at the next planning.

- Emergency exceptions require PO + team agreement **and** removing an equivalent item
- Why? It protects the team's focus and makes velocity measurement meaningful
- Constant scope changes → unpredictable velocity → impossible forecasting

“The sprint is a short-term contract between the team and the Product Owner.”

Sprint burndown chart

YOUR PROJECT



- X-axis: sprint days · Y-axis: points or hours remaining
- **Ideal line:** straight from the day-1 total to zero on the final day
- **Actual line:** real remaining work each day
- Above ideal → behind schedule; below ideal → ahead

- - - ideal

— actual

Used for daily inspection and early warning.

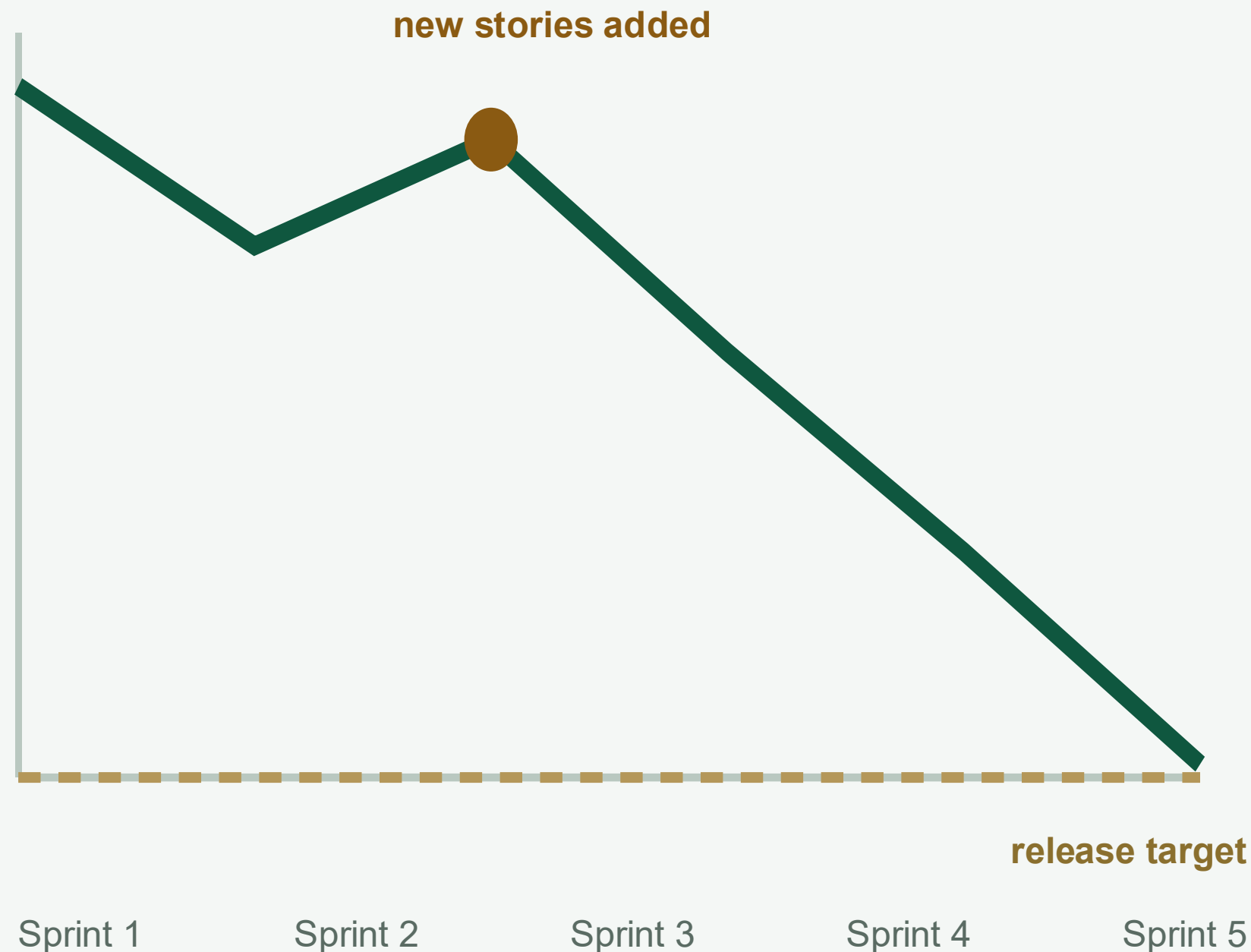
Which units? Hours historically. Points commonly, today. For a small team, item count — the milestone bar gives it free.

A flat burndown usually means scope was added — and the chart cannot show that. A burn *up* can.

Release burndown

YOUR PROJECT

STORY POINTS REMAINING IN THE PRODUCT BACKLOG



Progress across the whole release — not one sprint.

- Sprints across · points remaining in the product backlog up
- Ideal: a steady decline toward the release target
- Reality: bumpy — stories get added mid-release

Pair it with a burn-up. A burndown cannot tell “we are slow” from “scope grew” — both flatten the line. A burn-up draws scope separately, so growth shows.

Forecast a range, not a date. Extend trend lines from your best and worst recent sprints.

This is the stakeholder-facing chart. The sprint burndown is team-internal.

AI in project management

INDUSTRY

OPPORTUNITIES

- **The review bottleneck moves** — more generated code, the same reviewer attention. The constraint shifts from writing to reviewing
- **AI as a backlog participant** — agents can own and close items, with tracking and human sign-off
- **Backlog hygiene** — triage, deduplication, labelling, stale-issue detection, progress summaries

RISK

Output capacity grows; human review capacity does not. Oversight becomes the bottleneck.

An AI estimate is a second opinion, never the first number. Points are relative to *your* calibration, and the discussion is the point.

AI estimate



team discussion



final estimate

AI suggests — humans confirm.